Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – разработка ассемблерной вставки»

Выполнил:
Студент 4-го курса
Группы АС-66
Кацевич А.Ю.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками.

Ход работы

Вариант 6

6. Нечетные символы заменить на символ +.

Задание: написать ассемблерную вставку, реализующую обработку строки по варианту, и оформить её в виде отдельной функции; реализовать ту же обработку на C++ и сравнить быстродействие обоих вариантов.

Нечётными считаются символы на позициях 1, 3, 5, … (позиции считаются с единицы), то есть элементы строки с индексами 0, 2, 4, …. Например, из строки «HelloWorld» получается «+e+l+W+r+d».

Платформа. Работа выполнена на Mac с процессором Apple (архитектура ARM64): Visual Studio и вставки __asm для x86 на этом компьютере недоступны. Поэтому ассемблерная вставка записана командами ARM64 в синтаксисе Clang (asm volatile). Для Mac с процессором Intel в той же функции есть ветка на командах x86-64; она проверена запуском через Rosetta (6029 строк, 0 ошибок), но время измерялось только на ARM64.

Алгоритм вставки: в регистр w3 заносится код символа «+» (43); в цикле команда strb записывает его по адресу p и сразу увеличивает p на 2; счётчик n уменьшается на 2, цикл повторяется, пока n > 0. Длина строки передаётся в n из функции strlen. Для пустой строки вставка не выполняется.

Текст программы (lab1.cpp)

```
// Лабораторная работа №1, вариант 6.
// Задание: нечётные символы строки заменить на символ '+'.
// Нечётными считаются символы на позициях 1, 3, 5, ... (позиции считаем с единицы),
// то есть элементы массива с индексами 0, 2, 4, ...
// Реализация: функция на C++, функция с ассемблерной вставкой, сравнение скорости.

#include <algorithm>
#include <chrono>
#include <cstdlib>
#include <cstring>
#include <iostream>
#include <string>
#include <vector>

using namespace std;

// ---------- Вариант на C++ ----------
void replaceOddCpp(char* str) {
    int len = strlen(str);
    for (int i = 0; i < len; i += 2) {
        str[i] = '+';
    }
}
```

```
// ---------- Вариант с ассемблерной вставкой ----------
// Mac с процессором Apple (ARM64): вставка в синтаксисе Clang.
// Для Mac с Intel ниже есть вторая ветка на командах x86-64.
void replaceOddAsm(char* str) {
   long len = strlen(str);
   if (len == 0) return;

#if defined(__aarch64__)
   asm volatile(
      "mov w3, #43\n"            // w3 = 43 = код символа '+'
      "1:\n"
      "strb w3, [%[p]], #2\n"    // записать '+' по адресу p, затем p = p + 2
      "subs %[n], %[n], #2\n"    // n = n - 2, флаги выставляются по результату
      "b.gt 1b\n"                // пока n > 0, повторять цикл
      : [p] "+r"(str), [n] "+r"(len)
      :
      : "x3", "cc", "memory");
#elif defined(__x86_64__)
   asm volatile(
      "1:\n"
      "movb $43, (%[p])\n"       // записать '+' по адресу p
      "addq $2, %[p]\n"          // p = p + 2
      "subq $2, %[n]\n"          // n = n - 2
      "jg 1b\n"                  // пока n > 0, повторять цикл
      : [p] "+r"(str), [n] "+r"(len)
      :
      : "cc", "memory");
#else
#error "Нужен Mac (ARM64 или x86-64) и компилятор Clang"
#endif
}

// ---------- Проверка правильности ----------
// Эталон строится прямо по условию задачи, без циклов с шагом 2.
string expected(const string& s) {
   string r = s;
   for (size_t i = 0; i < r.size(); i++) {
      if (i % 2 == 0) r[i] = '+';
   }
   return r;
}

// Прогоняет обе функции на одной строке и сравнивает с эталоном.
bool checkOne(const string& s) {
   char a[2048], b[2048];
   strcpy(a, s.c_str());
   strcpy(b, s.c_str());
   replaceOddCpp(a);
   replaceOddAsm(b);
   string want = expected(s);
   return want == a && want == b;
}

int runTests() {
   int failed = 0, total = 0;

   // Фиксированные случаи: пустая строка, 1, 2, 3 символа, пробелы, цифры.
   const char* fixed[] = {"", "a", "ab", "abc", "HelloWorld", "a b c d", "12345", "  ", "x\ty"};
   for (const char* s : fixed) {
      total++;
      if (!checkOne(s)) {
         failed++;
         cout << "ОШИБКА на строке: \"" << s << "\"\n";
      }
   }
```

```
    // Все длины от 0 до 300 и случайные печатные символы.
    srand(12345);
    for (int len = 0; len <= 300; len++) {
        for (int k = 0; k < 20; k++) {
            string s;
            for (int i = 0; i < len; i++) s += (char)(33 + rand() % 94);
            total++;
            if (!checkOne(s)) {
                failed++;
                cout << "ОШИБКА на строке длины " << len << "\n";
            }
        }
    }

    cout << "Проверено строк: " << total << ", ошибок: " << failed << "\n";
    return failed == 0 ? 0 : 1;
}

// ---------- Измерение времени ----------
// Подставляем этот барьер после каждого вызова, чтобы компилятор не выбросил
// вызов функции как «бесполезный».
inline void barrier(char* p) {
    asm volatile("" : : "r"(p) : "memory");
}

long long measureCpp(char* buf, int iterations) {
    auto start = chrono::steady_clock::now();
    for (int i = 0; i < iterations; i++) {
        replaceOddCpp(buf);
        barrier(buf);
    }
    auto end = chrono::steady_clock::now();
    return chrono::duration_cast<chrono::nanoseconds>(end - start).count();
}

long long measureAsm(char* buf, int iterations) {
    auto start = chrono::steady_clock::now();
    for (int i = 0; i < iterations; i++) {
        replaceOddAsm(buf);
        barrier(buf);
    }
    auto end = chrono::steady_clock::now();
    return chrono::duration_cast<chrono::nanoseconds>(end - start).count();
}

// Один замер может «шуметь» из-за других программ, поэтому делаем 5 замеров
// и берём средний по величине (медиану).
long long median(vector<long long> v) {
    sort(v.begin(), v.end());
    return v[v.size() / 2];
}

void benchmark(const string& text, int iterations) {
    char bufCpp[4096], bufAsm[4096];
    strcpy(bufCpp, text.c_str());
    strcpy(bufAsm, text.c_str());

    vector<long long> timesCpp, timesAsm;
    for (int round = 0; round < 5; round++) {
        timesCpp.push_back(measureCpp(bufCpp, iterations));
        timesAsm.push_back(measureAsm(bufAsm, iterations));
    }
    long long tCpp = median(timesCpp);
    long long tAsm = median(timesAsm);

    cout << "Длина строки: " << text.size() << ", вызовов в одном замере: " << iterations << " (медиана из 5)\n";
```

```
    cout << "  C++:       " << tCpp / 1000000.0 << " мс (" << (double)tCpp / iterations << " нс на вызов)\n";
    cout << "  Ассемблер:  " << tAsm / 1000000.0 << " мс (" << (double)tAsm / iterations << " нс на вызов)\n";
    cout << "  Отношение C++ / ассемблер: " << (double)tCpp / tAsm << "\n";
}

// ---------- Главная функция ----------
int main(int argc, char* argv[]) {
    string mode = (argc > 1) ? argv[1] : "";

    if (mode == "--test") return runTests();

    cout << "Лабораторная работа №1, вариант 6\n";
    cout << "Нечётные символы строки (позиции 1, 3, 5, ...) заменяются на '+'.\n\n";

    // Строка: из аргумента "--demo" берём готовую, иначе просим ввести.
    string input = "HelloWorld";
    if (mode != "--demo") {
        cout << "Введите строку (только латиница, цифры и знаки, без русских букв): ";
        getline(cin, input);
    }

    // Один символ должен занимать один байт, иначе позиции собьются.
    for (unsigned char c : input) {
        if (c >= 128) {
            cout << "Ошибка: в строке есть символы не из ASCII (например, русские буквы).\n";
            return 1;
        }
    }
    if (input.size() >= 2000) {
        cout << "Ошибка: строка слишком длинная (максимум 1999 символов).\n";
        return 1;
    }

    char strCpp[2048], strAsm[2048];
    strcpy(strCpp, input.c_str());
    strcpy(strAsm, input.c_str());

    replaceOddCpp(strCpp);
    replaceOddAsm(strAsm);

    cout << "Исходная строка:      " << input << "\n";
    cout << "Результат C++:       " << strCpp << "\n";
    cout << "Результат ассемблера: " << strAsm << "\n";
    cout << (strcmp(strCpp, strAsm) == 0 ? "Результаты совпадают.\n" : "Результаты РАЗЛИЧАЮТСЯ!\n");

    cout << "\n=== Сравнение быстродействия ===\n";
    benchmark("HelloWorld", 10000000);
    benchmark(string(1000, 'a'), 1000000);

    return 0;
}
```

Результаты работы

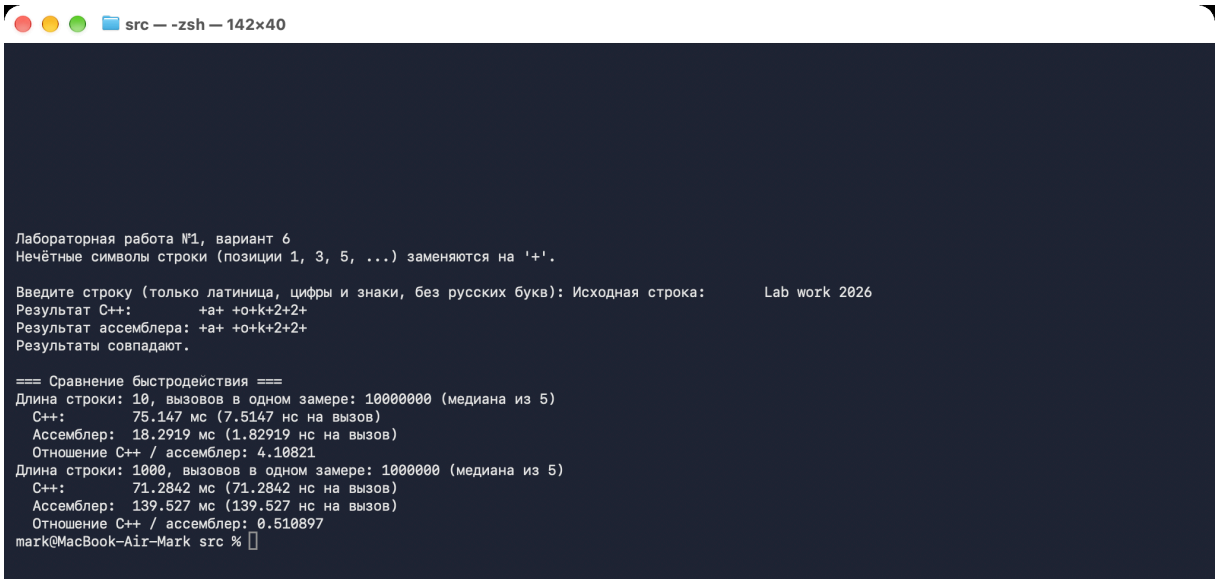


Рисунок 1 – Запуск программы: результат обеих функций и сравнение быстродействия (-O2)



Рисунок 2 – Автоматическая проверка: 6029 строк, ошибок нет

Таблица 1 – Время одного вызова при оптимизации -O2 (медиана из 5 замеров)

Длина строки, символов	C++, нс на вызов	Ассемблер, нс на вызов	C++ / ассемблер
10	7,45	2,08	3,58
1000	71,19	138,48	0,51

Таблица 2 – Время одного вызова без оптимизации -O0 (медиана из 5 замеров)

Длина строки, символов	C++, нс на вызов	Ассемблер, нс на вызов	C++ / ассемблер
10	4,43	2,74	1,62
1000	250,54	139,69	1,79

Вывод: изучил программную модель микропроцессора, регистры и организацию памяти; практически закрепил работу с ассемблерными вставками. Функция на C++ и функция с вставкой дали одинаковый результат на всех 6029 проверенных строках.

По быстродействию: без оптимизации (-O0) вставка быстрее в 1,61636 раза на строке из 10 символов и в 1,79353 раза на строке из 1000 символов. С оптимизацией (-O2) на короткой строке вставка быстрее в 3,58218 раза, а на длинной строке C++ быстрее: отношение C++ / ассемблер равно 0,514131. В машинном коде функции на C++ видно, что компилятор развернул цикл и записывает по два символа за проход, а в моей вставке за проход записывается один символ. Значит, ассемблерная вставка не гарантирует выигрыша: оптимизирующий компилятор может дать код не хуже.